STEM Ranked

Abstract

This is a ranking system for STEM students at I.M. Terrell based on their performance in math and science classes, computer science projects, and weighted GPA, certificates, and extracurricular activities. There will also be an overall ranking based on your scores in every other category, determining the top STEM majors. Eventually, it could be implemented into our school's core award system every 6 weeks.

Table of Contents

Abstract.....1
Table of Contents.....1
Introduction.....1
Overview.....2
Research.....2
Coding.....2
Bugs.....8
6.0 Future Work.....8
7.0 References.....8

Introduction

(October 20, 2025)

I actually started this project months prior with an idea: STEM majors seem to always be neglected at my school, and with no competitive drive, nothing was pushing us forward to be better. I wanted to create something that motivated the rest of the STEM department at I.M. Terrell to push forward and be the best, and hopefully leave a legacy for myself at this school when I inevitably leave next year. This idea became a concept of a ranking system, similar to something you would see in competitive spaces like Minecraft's PVP ranking system on mctiers.com.

I started by making a Google Form connected to a Google Sheets, with the idea that users could upload their accomplishments to me, ranked accordingly based on a specific criterion. The ranking list would update every month, giving people time to create new projects to blast themselves up on the leaderboard. I envisioned every Monday, when reviewing school-wide slideshows, a ranking system being presented in front of the student body that would motivate STEM students to rise to the top.

I lost interest in this project a little while later because I couldn't find a way to create a website unblocked by my school district, until now. My new goal is to use p5.js to use my HTML, CSS, and JavaScript(JS) skills to create a website that will display our school's top STEM

Overview

(October 20, 2025)

This project aims to create a ranking system for STEM students at I.M. Terrell based on their performance in math and science classes, computer science projects, and weighted GPA, certificates, and extracurricular activities. There will also be an overall ranking based on your scores in every other category, determining the top STEM majors.

Ranking will be done by either a team of moderators I appoint, a club, or myself. The I.M. Terrell STEM ranking system is based on merit and overall performance. You can only resubmit this form every month. Once your answers are flagged for review and one of our appointed moderators will check your submission and update your rank accordingly. You cannot go down in rank unless your new submission to a category is worse than your last one, or someone submits a better project than you for a category. You will be placed in a different tier depending on the level of merit shown in your submission. There can only be one HT1 and two LT1s for each category. In most categories, you will submit a screenshot. You can submit multiple items in different coding languages for categories like Computer Science. Projects for people in the top 25 in a category will be shown on the website. The top 10 in each category will potentially need to be fact-checked on responses—they will have to provide documentation. More categories will be added in the future.

Research

Coding

Entry 1.0

(October 21, 2025)

I started by adding my school's logo with a link to their website in the top right corner of my website. I uploaded the logo to a file called assets and added it to my HTML file, then added the file parameters, and linked the website in my HTML file (Fig. 1.1). After, I changed the title and added the icon to the website image as well (Fig. 1.2).

```
        <a href="https://imterrell.fwisd.org/"
target="_blank">
          <img src="assets/IMT.png" alt="Terrell logo"
style="width: 40px;height: 40px;">
        </a>
```

Fig. 1.1 logo and link to IMT page

```
    <title>STEM Ranked</title>
    <link rel="icon" type="image/x-icon"
href="/assets/IMT.ico">
```

Fig. 1.2 this would change the tab name and the tab icon

I wanted to add a unique background that was a video. I couldn't find any videos that I could install on my school-issued MacBook, because of the amount of district-wide blocks. I found one on Canva and uploaded it to my assets folder. I started by making a new class in the body called main_body, where I could put all of my main elements. I referenced a video and made sure it played automatically when the site is launched and looped (Fig. 1.3). Then, I edited the CSS file to adjust some of the parameters for the video (Fig.1. 4). I'll adjust the size later and change the speed at which it scrolls.

```
  <body>
    <div class="main_body">
      <video autoplay loop muted playsinline class="back-
video">
        <source
        src="assets/STEM_Ranked_Background_1.mp4"
        type="video/mp4">
      </video>
```

Fig. 1.3 this plays a video I have in my assets of just a blue background

```
.back-video {
  position: fixed;
  width:100%;
  height: 100%;
  z-index: -1;
}
```

Fig. 1.4 I am actively working on fixing the class to fit the whole screen

I added a container class for the main area where it would display all the rankings. I then added subsequent courses for the actual individual rank displays: rank & card (Fig. 1.5). I went into my CSS file and set up the styling for the container. The important thing I did here was make the flex-direction column so that all of the elements in the container class would be set up in a column format (Fig. 1.6).

```
        <div class="container">
          <div class="rank">
            <div class="card">
              1.
```

Fig. 1.5 so far the class order

```
.container {
  display: flex;
  justify-content: flex-start;
  align-items: center;
  flex-direction: column;
  position: relative;
  margin: 0 auto;
  height: 300vh;
  width: 70%;
  background-color: #06132D60;
  padding: 5px 10px;
  border-width: 5px;
  border-style: solid;
  border-color: #091733;
  border-radius: 15px;
}
```

Fig. 1.6 the styling for the container class

When setting up the rank class, I added some effects when you hover over the element. Whenever you hover over it, it gets bigger and lights up a little (Fig. 1.7). I made the rank class so I could make subsequent rank objects with ease. I plan to implement a sound effect every time a button is hovered over, but that will be later.

```
.rank {
  display: flex;
  justify-content: flex-start;
  align-items: flex-start;
  width:100%;
  height: 30%;
  background-color: #0C1F44;
  border-width: 5px;
  border-style: solid;
  border-color: #091733;
  border-radius: 8px;
  cursor: pointer;
  transition: background-color 0.3s ease;
  margin-bottom: 10px;
}

.rank:hover {
  background-color: #06284D;
  transform: scale(1.02);
}
```

Fig. 1.7 rank class with hover effects

I added an image under the card class in the HTML file as a placeholder. Then I went back into the CSS file and made the shape a trapezoid by setting up a clip path (Fig. 1.8). This is currently what the website looks like (Fig. 1.9).

```
.card {
  width:25%;
  height: 100%;
  background-color: #FFEB3B;
  clip-path: polygon(0% 0%, 100% 0%, 85% 100%, 0% 100%);

}
```

Fig. 1.8 polygon is an upside-down trapezoid

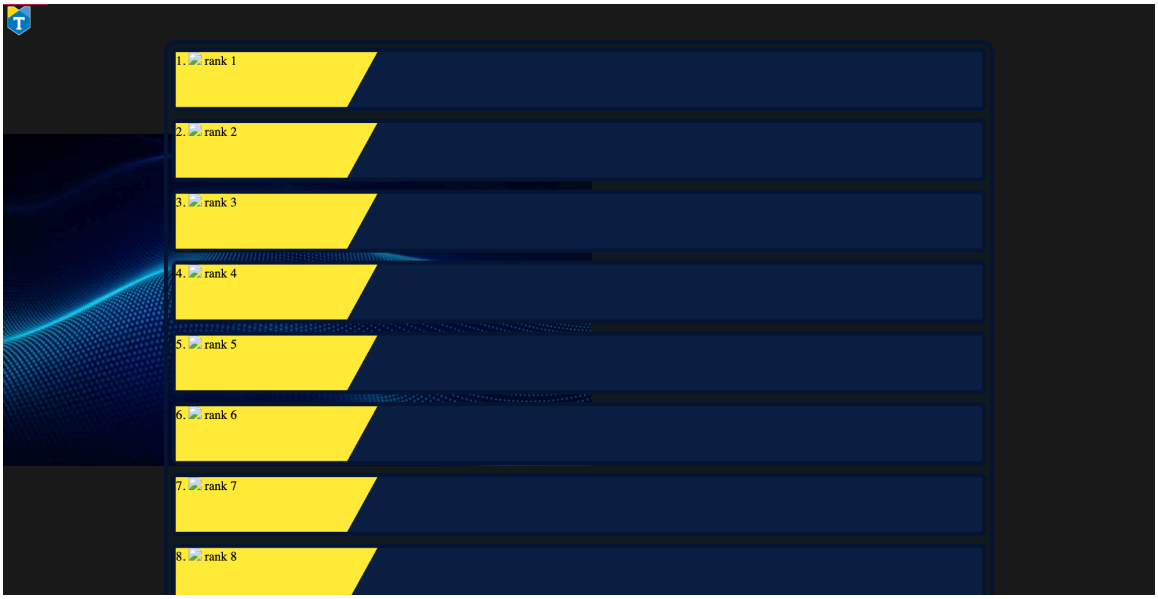

Fig. 1.9 current website on [p5.js]

After I fixed the issue of the background not working properly, I had a background that cut off after a certain point. So now I needed to make the video scroll with the user, but make it scroll slower so that it appears as if the image extends. I started by creating a script in my HTML file in JavaScript, that listens for whenever the user scrolls and then transforms the video by the scrollY multiplied by the scroll speed factor, which controlled how fast the background moved when scrolling (Fig. 1.10). Then I went into the CSS file, and made sure that it starts as 0 (Fig. 1.11).

```
<script>
    const backVideo = document.querySelector('.back-video');
    const scrollSpeedFactor = 0.5;

    window.addEventListener('scroll', () => {
        const scrollY = window.scrollY;
        backVideo.style.transform = `translateY(${scrollY * scrollSpeedFactor}px)`;
    });
</script>
```

Fig. 1.10 this is JS in an HTML file

```
.back-video {
  position: absolute;
  right: 0;
  bottom: 200;
  min-width:200%;
  min-height: 200%;
  width: auto;
  height: auto;
  z-index: -1;
  object-fit: cover;
  overflow: hidden;
  transform: translateY(0);
}
```

Fig. 1.11 added transform line at the bottom

Entry 2.0

(February 28, 2026)

Today, I want to update my HTML file, as it is kind of cluttered without much meaning. First, I decided to add comments, then I included the JS that would be responsible for parsing student data from an HTML spreadsheet (Fig. 1.12). Then, instead of making a tab for all 25 ranks, I just made a container that will automatically make the tabs instead (Fig. 1.13).

```
    </script>

    <!-- Ranking script -->
    <script src="parse.js"></script>

  </body>
</html>
```

Fig. 1.12 "parse.js," is the script responsible for sorting through student data

```
<!-- Dynamic Ranking Container -->
<div class="container" id="rankingContainer">
  <!-- Student Cards will be injected here by parse.js -->
</div>

<div class="footer"></div>
</div>
```

Fig. 1.13 the name cards will now display everything, based on data from the JS file

Next, I started making the actual JavaScript file. My first idea was to create a map with keys and values to assign each student's specific variables to a category. Then I created an ArrayList for the students (Fig. 1.14).

```
const CATEGORY_MAP = {
  "Overall": "overall",
  "Math": "math",
  "Science": "science",
  "Computer Science": "cs",
  "Weighted GPA": "gpa",
  "Certificates": "certificates",
  "Extra Curricular's": "extra"
};

let students = [];
```

Fig. 1.14 this will act as a lookup table/dictionary

Next began the actual HTML parsing. I decided to make the function asynchronous so it wouldn't interfere with other methods that control the page's display. Then, I defined the path to the HTML file and converted it to a string so that the DOMParser interface could convert it into a DOC file, and from there, I defined all the rows (Fig. 1.15).

The next step was to create a "for each " loop to cycle through all of the rows in the table. I had it skip the header at the top of the page and any empty cells. After the checks, I created a student object that is pushed into the students array, which has all of the information about the student from the table. Lastly, I safely retrieved the image data for the student's profile picture provided by the HTML table spreadsheet by having it return null if there is no image found (Fig. 1.16)

```
let students = [];

async function loadData() {
  const response = await fetch("data/Category_History.html");
  const text = await response.text();

  const parser = new DOMParser();
  const doc = parser.parseFromString(text, "text/html");
  const rows = doc.querySelectorAll("tbody tr");
```

Fig. 1.15 parses through the HTML table with the provided student information

```
  rows.forEach((row, index) => {
    if (index === 0) return;

    const cells = row.querySelectorAll("td");
    if (cells.length === 0) return;

    students.push({
      id: cells[0].innerText.trim(),
      grade: parseFloat(cells[1].innerText),
      last: cells[2].innerText.trim(),
      first: cells[3].innerText.trim(),
      math: parseFloat(cells[4].innerText),
      science: parseFloat(cells[5].innerText),
      cs: parseFloat(cells[6].innerText),
      gpa: parseFloat(cells[7].innerText),
      certificates: parseFloat(cells[8].innerText),
      extra: parseFloat(cells[9].innerText),
      profile: cells[13].querySelector("img")?.getAttribute("src") || null
    });
  });
}
```

Fig. 1.16 adds a student object to the student array with all the student's information

Now that we have all of the student information, it is time to automatically determine the ranking based on the average in every area. This was simple; the only thing that was needed was to create a function to return the student's overall average based on the given data (Fig. 1.17).

The next step was to determine the actual rank the student would be receiving. Every student category is ranked from 5-1, with each level having a low and high tier. In the spreadsheet, a high tier 1 is represented by the integer 1, while a low tier 1 is represented by the float 1.5. So the function needs to return High Tier if the number is an integer, and Low Tier if the number is a float (Fig. 1.18).

```
function calculateOverall(student) {
  return (
    (student.math +
      student.science +
      student.cs +
      student.gpa +
      student.certificates +
      student.extra) / 6
  );
}
```

Fig. 1.17, the HTML table has given values for a student's performance

```
function formatTier(value) {
  if (Number.isInteger(value)) {
    return `High Tier ${value}`;
  } else {
    return `Low Tier ${Math.floor(value)}`;
  }
}
```

Fig. 1.18 this method returns either high tier or low tier, depending on the category value

Now I need to actually render the cards. I start by making a new function to load any category. It starts by getting the category and removing all the elements from the container we made for all the cards previously. Then, it sorts through the ArrayList by putting high tiers on top and low tiers on the bottom (Fig. 1.19).

Finally, I go through every student in the newly sorted ArrayList, and I re-create their card for the website from before, and fill the container with the cards (Fig. 1.20). This allows for a changing number of people to be inserted on the website.

```
function renderCategory(categoryName) {
  const key = CATEGORY_MAP[categoryName];
  const container = document.getElementById("rankingContainer");
  container.innerHTML = "";

  let sorted = [...students];

  sorted.sort((a, b) => {
      const valA = key === "overall" ? calculateOverall(a) : a[key];
      const valB = key === "overall" ? calculateOverall(b) : b[key];
      return valA - valB;
  });
```

Fig. 1.19 it also quickly checks if the category is overall, and if it is, it runs the other function

```
  sorted.forEach((student, index) => {
    const value = key === "overall" ? calculateOverall(student) : student[key];

    const rankDiv = document.createElement("div");
    rankDiv.className = "rank";

    const card = document.createElement("div");
    card.className = "card";

    const profileImg = document.createElement("img");
    profileImg.className = "profile_picture";
    profileImg.src = student.profile || "assets/default_profile.png";

    const name = document.createElement("span");
    name.innerText = `${student.first} ${student.last}`;

    const tierBox = document.createElement("div");
    tierBox.className = "tier_box";
    tierBox.innerText = formatTier(value);

    const icon = document.createElement("img");
    icon.className = "category_icon";
    icon.src = "assets/placeholder_icon.png";

    card.innerHTML = `${index + 1}. `;
    card.appendChild(profileImg);
    card.appendChild(name);
    card.appendChild(tierBox);
    card.appendChild(icon);

    rankDiv.appendChild(card);
    container.appendChild(rankDiv);
  });
}
```

Fig. 1.20 this creates a card for every student in the sorted ArrayList

For the final few steps, I added a click event for the tabs that would change the category depending on the button press (Fig. 1.21). Lastly, I added comments to everything and made it start on the Overall tab whenever loading the data.

```
document.querySelectorAll(".tab").forEach(tab => {
    tab.addEventListener("click", () => {
        renderCategory(tab.innerText.trim());
    });
});

loadData();
```

Fig. 1.21 switches the category depending on the button pressed

Entry 3.0

I decided to update some of the icons so they appear on the website (Fig. 1.22). The next thing to do is try to fix the problem with some user icons not appearing on the website. I also desire to change the functionality of displaying the user's rank on the overall screen; it should be different than the other sections.

Fig. 1.22 all the users with the grey icon did not upload profile pictures to the form

Bugs

Entry 1.0

(October 21, 2025)

Whenever I was trying to configure the size of the background video in my CSS file, I kept running into issues making it fit the full screen and achieve the effect that I wanted. I tried various things, and nothing worked.

In the end, I fixed the issue by adding a minimum width and height, and adding an object fitting and hiding the overflow (Fig. 2.1).

```
.back-video {
  position: absolute;
  right: 0;
  bottom: 200;
  min-width:200%;
  min-height: 200%;
  width: auto;
  height: auto;
  z-index: -1;
  object-fit: cover;
  overflow: hidden;
```

Fig. 2.1 I chose not to make it a fixed position because I wanted it to scroll, just slightly

Entry 2.0

(February 28, 2026)

There were several issues with the HTML after I created the automatic tab system. So, I cleaned up the function and made sure to append the specific elements to the correct classes they are supposed to belong to (Fig. 2.2).

```
    rankDiv.className = "rank";

    const card = document.createElement("div");
    card.className = "card";

    const rankNumber = document.createElement("span");
    rankNumber.className = "rank_number";
    rankNumber.innerText = `${index + 1}.`;

    const profileImg = document.createElement("img");
    profileImg.className = "profile_picture";
    profileImg.src = student.profile || "assets/default_profile.png";

    const name = document.createElement("span");
    name.className = "student_name";
    name.innerText = `${student.first} ${student.last}`;

    const tierBox = document.createElement("div");
    tierBox.className = "tier_box";
    tierBox.innerText = formatTier(value);

    const icon = document.createElement("img");
    icon.className = "category_icon";
    icon.src = "assets/placeholder_icon.png";

    // Append elements to RANK (not card)
    rankDiv.appendChild(card); // background shape
    card.appendChild(rankNumber);
    card.appendChild(profileImg);
    rankDiv.appendChild(name);
    rankDiv.appendChild(tierBox);
    tierBox.appendChild(icon);

    container.appendChild(rankDiv);
  });
}
```

Fig. 2.2 I may rework the rest of the HTML again later.

6.0 Future Work

1. Create a sorting system that parses through a html file full of responses from students
2. Display items from the responses file on the website
3. Change the display depending on the selected tab
4. Go through the schools stem department for approval
5. Go through the school's administration for approval

7.0 References

- p5.js (https://p5js.org/) - the designated compiler that runs the code